



Figure 2: OpenGL commands on a Linux architecture

support for networked computers.

- The OpenGL Utility Toolkit (GLUT) is a Window System-independent toolkit, which is used to hide the complexities of differing window system APIs. We will be using GLUT in our sample program.
- Mesa helps in running OpenGL programs on an X Window System, especially Linux.
- OpenInventor is a C++ 3D graphics API that abstracts on OpenGL.

Now, let's see what the workflow of a C++ OpenGL program looks like. We have our application program, the compiled code at one end. At the other end is a 3D image that is rendered to the screen.

Our program might be written with OpenInventor commands (or in native C++). The function calls are declared in OpenGL. These OpenGL calls are platform-independent. On an X Window System (like Linux), the OpenGL calls are redirected to the implementation calls defined by Mesa. The operating system might also have its own rendering API at this stage, which accelerates the rendering process. The OpenGL calls might also redirect to the implementation calls defined by the proprietary drivers (like Nvidia drivers, Intel drivers, etc, that facilitate graphics on the screen). Now, these drivers convert the OpenGL function calls into GPU commands. With its parallelism and speed, GPU maps the OpenGL commands into 3D images that are projected onto the screen. And this way we finally see 3D images called by OpenGL.

Setting up Linux for graphics programming

For graphics programming on a Linux machine, make sure that OpenGL is supported by your graphics card—almost all of them do. OpenGL, GLX and the X server integration of GLX, are Linux system components and should already be part of the Debian, Red Hat, SUSE or any other distribution you use. Also make sure your graphics drivers are up to date. To update the graphics drivers on Ubuntu,

go to **System settings >> Software and Updates >>**

Additional drivers and install the required drivers.

Next, install GLUT. Download the GLUT source from http://www.opengl.org/resources/libraries/glut/glut_downloads.php and follow the instructions in the package. If you're programming in C++, the following files need to be included:

```
#include <GL/glut.h> // header file for the GLUT library
#include <GL/gl.h> // header file for the OpenGL32 library
#include <GL/glu.h> // header file for the GLU32 library
```

To execute this file, an OpenGL program, you require various development libraries like GL, GLU X11, Xrandr, etc, depending on the function calls used. The following libraries would be sufficient for a simple 3D graphics program. Use *apt-get* or *yum* to install these libraries:

```
freeglut3
freeglut3-dev
libglew1.5
libglew1.5-dev
libglu1-mesa
libglu1-mesa-dev
libgl1-mesa-glx
libgl1-mesa-dev
```

Next comes the linking part. You can use a *Makefile* to do this. But for our example, we will link them manually while compiling. Let's say we want to execute a C++ OpenGL program called *main.cpp*. We can link OpenGL libraries as follows:

```
g++ -std=c++11 -Wall -o main main.cpp -lglfw3 -lGLU -lGL
-lX11 -lXxf86vm -lXrandr -lpthread -lXi
```

Note that the linking is done sequentially. So the order is important. You can now execute the program the usual way (*/ main* for the above).

A simple OpenGL based example

Let's now write a minimal OpenGL program in C++. We will use GLUT for windowing and initialising context.

We have already discussed what an OpenGL based workflow looks like. We create a window, initialise a context, wait for some event to occur and handle it. For this simple example, we can ignore event handling. We will create a window and display a triangle inside it.

The required headers are:

```
#include <GL/glut.h> // header file for the GLUT library
#include <GL/gl.h> // header file for the OpenGL32 library
```

The latest versions of GLUT include *gl.h*. So we can only include the GLUT library header.